

# Git Cheat Sheet

The everyday command reference for local work and collaboration.

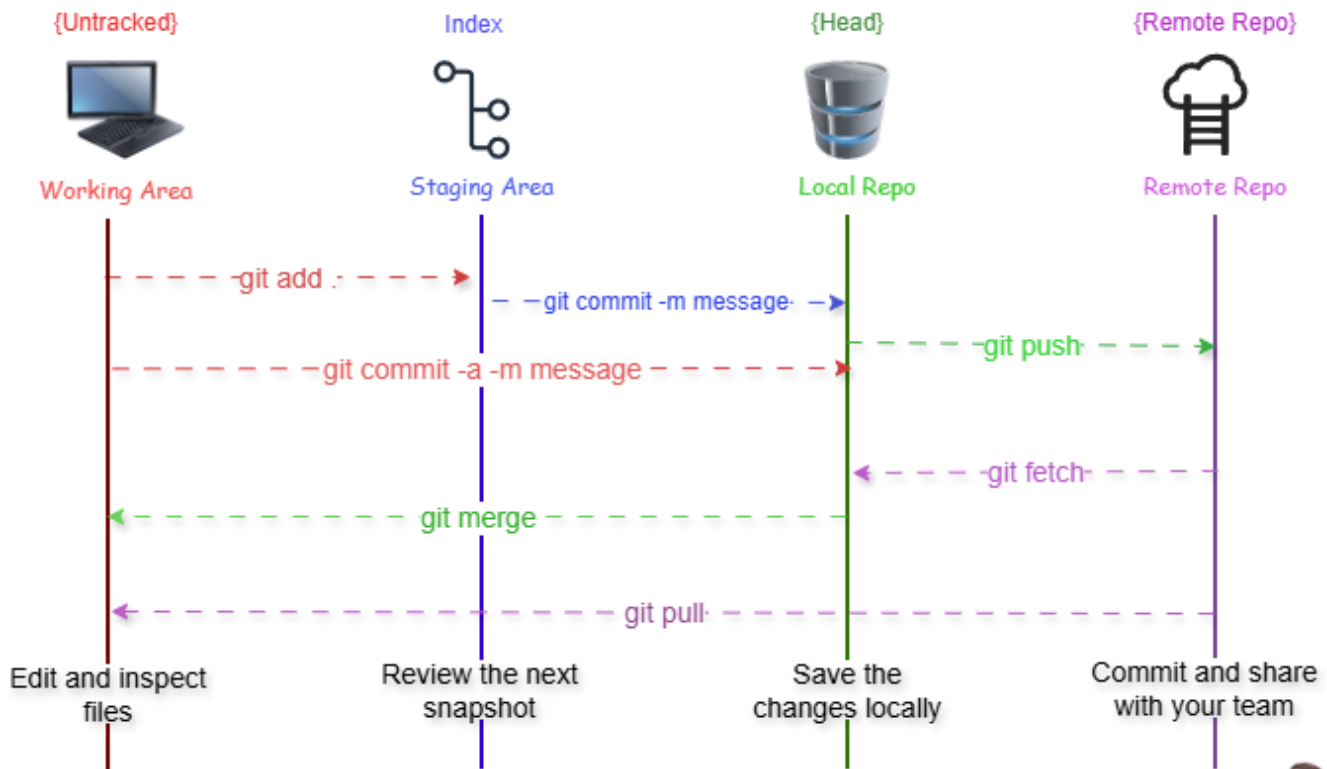


## FIRST-TIME SETUP

|                                                               |                   |
|---------------------------------------------------------------|-------------------|
| <code>git config --global user.name "Your Name"</code>        | Set your name     |
| <code>git config --global user.email "you@example.com"</code> | Set your email    |
| <code>git config --list</code>                                | View all settings |

## START HERE

|                                    |                          |
|------------------------------------|--------------------------|
| <code>git init</code>              | Create a repository      |
| <code>git clone &lt;url&gt;</code> | Copy a remote repository |
| <code>git log --oneline</code>     | View compact history     |
| <code>git status</code>            | Inspect current changes  |



TechieNicks.com

# Git Cheat Sheet

The everyday command reference for local work and collaboration.

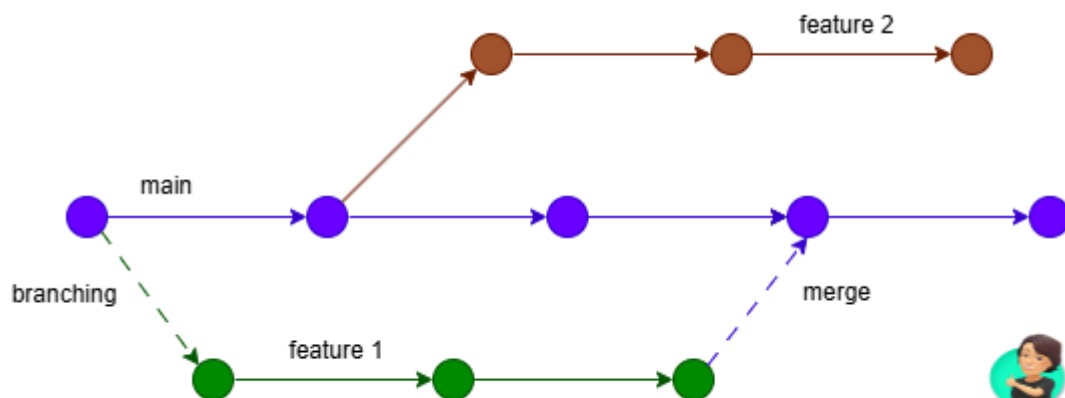


## STAGE & COMMIT

|                                       |                                            |
|---------------------------------------|--------------------------------------------|
| <code>git add &lt;file&gt;</code>     | Stage one file                             |
| <code>git add .</code>                | Stage all changes                          |
| <code>git add -p</code>               | Stage interactively (hunk by hunk)         |
| <code>git diff</code>                 | View unstaged changes                      |
| <code>git diff --cached</code>        | View staged changes                        |
| <code>git commit -m "message"</code>  | Save a snapshot                            |
| <code>git commit -am "message"</code> | Stage tracked files and commit in one step |
| <code>git commit --amend</code>       | Edit the last commit message or content    |

## BRANCHING

|                                             |                                             |
|---------------------------------------------|---------------------------------------------|
| <code>git branch</code>                     | List all local branches                     |
| <code>git branch &lt;name&gt;</code>        | Create a new branch                         |
| <code>git branch -d &lt;name&gt;</code>     | Delete a branch safely                      |
| <code>git branch -D &lt;name&gt;</code>     | Force delete a branch                       |
| <code>git checkout &lt;branch&gt;</code>    | Switch to an existing branch                |
| <code>git checkout -b &lt;branch&gt;</code> | Create and switch to a new branch           |
| <code>git switch &lt;branch&gt;</code>      | Switch branches (modern command)            |
| <code>git switch -c &lt;branch&gt;</code>   | Create and switch branches (modern command) |



# Git Cheat Sheet

The everyday command reference for local work and collaboration.



## MERGING & REBASING

|                                             |                                                |
|---------------------------------------------|------------------------------------------------|
| <code>git merge &lt;branch&gt;</code>       | Merge specified branch into the current branch |
| <code>git rebase &lt;branch&gt;</code>      | Rebase current branch onto specified branch    |
| <code>git rebase -i HEAD~3</code>           | Interactive rebase for the last 3 commits      |
| <code>git rebase --continue</code>          | Continue rebase after resolving conflicts      |
| <code>git rebase --abort</code>             | Cancel the rebase operation                    |
| <code>git cherry-pick &lt;commit&gt;</code> | Apply a specific commit from another branch    |

## SYNCING with REMOTE

|                                                |                                                       |
|------------------------------------------------|-------------------------------------------------------|
| <code>git remote -v</code>                     | List all configured remotes                           |
| <code>git remote add origin &lt;url&gt;</code> | Add a new remote repository                           |
| <code>git fetch</code>                         | Download changes from remote without merging          |
| <code>git pull</code>                          | Download and merge remote changes into current branch |
| <code>git pull --rebase</code>                 | Fetch changes and rebase current branch               |
| <code>git push</code>                          | Upload local commits to remote                        |
| <code>git push -u origin &lt;branch&gt;</code> | Push and set upstream tracking branch                 |
| <code>git push --force-with-lease</code>       | Force push safely without overwriting unknown work    |

## INSPECTING HISTORY & DIFFS

|                                                    |                                            |
|----------------------------------------------------|--------------------------------------------|
| <code>git log</code>                               | View full commit history                   |
| <code>git log --oneline</code>                     | View compact single-line history           |
| <code>git log --oneline --graph --all</code>       | View a compact visual tree of all branches |
| <code>git show &lt;commit&gt;</code>               | View details and diff of a specific commit |
| <code>git diff</code>                              | View unstaged changes                      |
| <code>git diff --staged / git diff --cached</code> | View staged changes                        |
| <code>git blame &lt;file&gt;</code>                | Show who modified each line of a file      |

# Git Cheat Sheet

The everyday command reference for local work and collaboration.



## UNDOING CHANGES

|                                                |                                                    |
|------------------------------------------------|----------------------------------------------------|
| <code>git restore &lt;file&gt;</code>          | Discard unstaged changes in working directory      |
| <code>git restore --staged &lt;file&gt;</code> | Unstage a file while keeping changes               |
| <code>git reset --soft HEAD~1</code>           | Undo commit, keep changes staged                   |
| <code>git reset --mixed HEAD~1</code>          | Undo commit, keep changes unstaged                 |
| <code>git reset --hard HEAD~1</code>           | Undo commit and discard all changes                |
| <code>git revert &lt;commit&gt;</code>         | Create a new commit that reverts a previous commit |

## STASHING

|                              |                                                |
|------------------------------|------------------------------------------------|
| <code>git stash</code>       | Save uncommitted changes temporarily           |
| <code>git stash list</code>  | View list of all stashed changes               |
| <code>git stash pop</code>   | Apply the latest stash and remove it from list |
| <code>git stash apply</code> | Apply the latest stash without removing it     |
| <code>git stash drop</code>  | Delete a specific stash                        |

## TAGS

|                                           |                               |
|-------------------------------------------|-------------------------------|
| <code>git tag</code>                      | List all tags                 |
| <code>git tag v1.0</code>                 | Create a lightweight tag      |
| <code>git tag -a v1.0 -m "message"</code> | Create an annotated tag       |
| <code>git push origin v1.0</code>         | Push a specific tag to remote |
| <code>git push origin --tags</code>       | Push all local tags to remote |

## CLEANING & MAINTENANCE

|                            |                                              |
|----------------------------|----------------------------------------------|
| <code>git clean -n</code>  | Preview untracked files that will be removed |
| <code>git clean -fd</code> | Remove untracked files and directories       |
| <code>git gc</code>        | Run garbage collection & optimize repository |
| <code>git fsck</code>      | Verify repository integrity                  |

# Git Cheat Sheet

The everyday command reference for local work and collaboration.



## SEARCHING

|                                      |                                                  |
|--------------------------------------|--------------------------------------------------|
| <code>git grep "text"</code>         | Search working tree for text string              |
| <code>git log -S "text"</code>       | Find commits that added or removed specific text |
| <code>git log --author="name"</code> | Filter commit log by author                      |

## Setting Up a GitHub Repository for the First Time

Two common paths:

- A. starting fresh (**new project**) or
- B. connecting an existing local project to GitHub.

### Path A: Create Repo on GitHub First, Then Clone

1. Create the Repository on GitHub
  - a. Go to <https://github.com/new>
  - b. Enter repo name, (description is optional)
  - c. Choose **Public** or **Private**
  - d. Optionally check "Add a README file"
  - e. Click **\*\*Create repository\*\***

Or via CLI:

```
gh repo create my-project --public --clone
```

(The `--clone` flag pulls it to your machine immediately.)

2. Clone It Locally

```
git clone https://github.com/username/my-project.git  
  
cd my-project
```

# Git Cheat Sheet

The everyday command reference for local work and collaboration.



## 3. Start Working

```
git add .  
git commit -m "Initial commit"  
git push
```

## Path B: You Already Have a Local Project — Push It to GitHub

### 1. Initialize Git in Your Project Folder

```
cd my-existing-project  
git init
```

### 2. Create a `.gitignore` (recommended)

Exclude files you don't want tracked (e.g., `node\_modules/`, `.env`, `venv/`).

```
touch .gitignore
```

GitHub has templates for common languages: [github.com/github/gitignore](https://github.com/github/gitignore)

### 3. Stage and Commit Your Files

```
git add .  
git commit -m "Initial commit"
```

### 4. Create the Repo on GitHub (empty, no README)

```
gh repo create my-project --public --source=. --remote=origin
```

Or manually on [github.com/new](https://github.com/new) — **don't** initialize with a README if you already have local files (avoids merge conflicts).

# Git Cheat Sheet

The everyday command reference for local work and collaboration.



## 5. Connect Local Repo to GitHub (if not done via `gh`)

```
git remote add origin https://github.com/username/my-project.git
```

## 6. Set the Default Branch Name

```
git branch -M main
```

## 7. Push Your Code

```
git push -u origin main
```

The `-u` sets `origin main` as the default upstream, so future pushes just need `git push`.

## Verify It Worked

From command line:

```
git remote -v      # should show origin pointing to your GitHub URL
git status         # should say "up to date with origin/main"
```

Or just visit the repo URL in your browser — your files should appear.

## Quick Reference — New Local Project to GitHub

```
git init
git add .
git commit -m "Initial commit"
gh repo create my-project --public --source=. --remote=origin --push
```

This one-liner (with `gh`) creates the repo, links it, and pushes in a single step.